Zebra CV Developer II — Team Interview Guide (28 Oct)

Oct 20, 2026 · @Nishal

What to expect on 28 Oct

Confirmed: a 90-minute virtual interview on Wednesday 28 Oct, 1:15–2:45 pm ET, with three people. It has two purposes: the team learns about you and your experience, and you meet the team and learn about the role. It **includes a brief C++ technical test**. Zebra calls this Round 1, so expect at least one more round after it.

Who	Title (from the invite)	What they will likely focus on
Jean-Sebastien Lemieux	Supervisor, Computer Vision Software Development	Probably your direct manager and the likeliest person to run the C++ test. Day-to-day engineering: C++ fluency, how you write, test and ship code
Sylvain Chapleau	Senior Manager, Computer Vision & Artificial Intelligence	Leads the wider CV and AI group. Big picture: your background, motivation, deep-learning direction, team fit, level
Paula Espinosa	Temporary/Contractor	Probably talent acquisition: process, timing, next steps, possibly salary

All three addresses are @zebra.com, which matches Zebra's official recruiting channel. The next section covers what public records say about each person and the questions that predicts.

The brief C++ test is the one part you can fully prepare for. Section 3 has a test drill at the top. The rest of the session is about your experience and the role, so sections 2, 6 and 7 matter as much as the technical sections.

What past candidates report (anonymous, small samples, from search results; not confirmed for this team):

- Zebra computer vision engineer: a phone screen, then a **presentation on a CV topic**, then rounds on CPU vs GPU implementation ("how would you do this on a GPU with CUDA?") and "given an algorithm, describe your process to turn it into production-worthy code."
- Matrox Montreal, before Zebra bought it: a **~45-minute written C++ and image-processing test** that goes from easy to hard and that "you are not expected to finish," then a 3-on-1 behavioural panel. A 2021 Montreal candidate got a take-home on references and OOP, then a technical interview on **debugging and pointers** (shared_ptr, class vs object).
- Zebra software roles generally: live coding (data structures, OOP), a project walkthrough and some system design, sometimes a longer loop with four to six engineers and managers.

Likely shape of the 90 minutes. The order is a guess, and the test may come first or last. Use it to pace yourself.

Time (ET)	Block	Your goal
1:15–1:25	Introductions; they present the team and the role	Note their current priorities and refer back to them later
1:25–1:50	Your experience	2-minute pitch, then the care-label QC deep-dive in 5 minutes with numbers (sections 2 and 6)
1:50–2:20	Brief C++ test	Fast, precise answers on basics; think aloud on harder items (section 3)
2:20–2:35	More on the role; behavioural questions	2–3 STAR stories
2:35–2:45	Your questions	One for each person (section 7)

Panel background and the questions it predicts

The two engineers are long-serving Matrox people with patents in areas the job description lists: Sylvain in **OCR and image classification**, Jean-Sébastien in **3D profiling**. Expect follow-ups to drift toward those topics, so prepare a solid 60-second answer for each (below). These come from public patent records and a data-aggregator profile; check LinkedIn to confirm.

Person	What public records show	Questions this predicts
Sylvain Chapleau, Senior Manager, CV & AI	LinkedIn in Laval, QC. ZoomInfo lists him as Manager, Software Engineering at Matrox in Dorval, dated to 1999 (aggregator, unverified). A colleague's recommendation cites "expertise in computer vision, programming and mathematics." Co-inventor on US 8,787,660, automatic font definition (filed 2005, OCR) and US 8,620,078, determining a class associated with an image (filed 2010); more on Justia.	How did your OCR handle many fonts and languages? How would you teach a reader a new font from a few samples? When do you choose classical methods over deep learning for classification? How do you train with few labelled images? Explain the maths behind a method you published. Where do GenAI and VLMs fit in industrial vision?
Jean-Sébastien Lemieux, Supervisor, CV Software Development	Co-inventor, with Vincent Zalzal, Christopher Hirst and Steve Massicotte, on US 11,120,294, 3D profiling of an object (sheet-of-light method, priority 2016) and its continuation US 11,908,173. One search result also names him on US 12,322,128, photometric stereo reconstruction; I couldn't confirm that. No public profile found. Probably your future manager.	How does laser triangulation work, and how do you calibrate it? How do you find the laser line at sub-pixel precision? What breaks it (shiny parts, occlusion)? What is photometric stereo? Plus the C++ test, your code-review and testing habits, and how you'd port a prototype
Paula Espinosa, Temporary/Contractor	Nothing public found.	Process and logistics: availability, relocation, possibly salary. Keep the same anchor (section 7)

Context worth knowing. Zebra funds an industrial research chair in computer vision at ÉTS Montréal (LIVIA lab). It works on inspection, object pose estimation, OCR and barcode reading, with a focus on deep learning that needs **less labelled data** and adapts

to changing conditions. Sylvain's CV & AI group is the likely industrial partner, though I couldn't confirm that. Few labels and drift are probably live topics for this group, and your synthetic-data work is a direct answer to the first.

Answers to prepare for these topics (60 seconds each)

- **Laser triangulation (sheet of light).** A laser line is projected onto the part and a camera views it at an angle. In each image column, find the line's peak row at sub-pixel precision: centre of gravity or a Gaussian/parabola fit over the intensity profile. Undistort, then intersect each pixel's camera ray with the calibrated laser plane to get a 3D point. Moving the part on an encoder sweeps out a full profile. Calibration needs the camera intrinsics plus a laser-plane fit from a target at several known heights. Failure modes: laser speckle, shiny surfaces and multiple reflections, and occlusion of the camera's or the laser's view.
- **Photometric stereo.** A fixed camera captures the part under 3 or more known light directions. Under a Lambertian assumption, $I = \rho\,(n \cdot l)$, so least squares per pixel gives the albedo and the surface normal. Integrate the normals (for example Frankot–Chellappa) to get relative height. It's good for embossed text, scratches and dents. It fails on specular highlights and shadows, so those pixels must be detected and rejected, or the method combined with another 3D source.
- **OCR for a new font from few samples.** Classical: segment the characters, normalise their size, extract features, and match them against font models learned from a sample string, with syntax rules to verify. Deep learning: a CRNN with CTC loss, trained on synthetic text rendered in that font plus augmentation (blur, contrast, perspective). Bridge to your work: multilingual care-label verification and reading digits off medical monitors.
- **Few labelled images.** Anomaly detection trained only on good parts; synthetic data; transfer learning from pre-trained backbones; active learning to choose what to label next. Your evidence: the rule-based generator that produced about 10,000 synthetic variations per pattern for your JAES model, and the training-free SSIM detector (95% detection).

How to use this in the room. Talk about the technology, not their patents: "I know Matrox has done sheet-of-light 3D profiling" is fine, but quoting someone's patent at them can feel like surveillance. Good questions that draw on it:

- To Sylvain: "How is the CV & AI group balancing classical tools and deep learning in Aurora, and does the team work with the ÉTS chair?"
- To Jean-Sébastien: "How much of the team's work is 2D library code versus 3D, now that Photoneo is part of Zebra?"

Your pitch and fit

The recruiter already accepted you. Engineers want proof that you can turn a vision algorithm into fast, tested C++ that runs on their hardware. Lead every answer with MAS industrial vision and use the audio work as proof that you still ship on hardware.

2-minute pitch for an engineering audience

> I'm an ML engineer with about ten years in industry and research. The constant has been taking detection algorithms from prototype to real-time code on constrained hardware. At MAS Holdings I spent five and a half years building production vision systems. One was a care-label inspection system that went from flatbed scanner, to industrial camera, to a conveyor with PLC reject, and cut inspection from 15–20 minutes per label to 1–2 minutes per batch. Another was loom-side fabric defect detection with a microscopic camera array that had to alert within 5–6 seconds. The third was the camera-to-print alignment algorithm and RIP integration behind Promptly, a garment-printing product now running in four countries. In my PhD and postdoc I did the same job for audio. My model runs in 14 ms at 31% CPU on a Raspberry Pi 4, 74 times faster than the baseline, and I recently wrote Nebula, a C++17 feature-extraction library with per-feature ARM benchmarks. What draws me here is that your team has shipped industrial vision software since the Matrox days, and that's the work I want to be back in.

Fit map: what the panel will probe

JD area	Your best evidence	Strength	How to play it
Prototype to product C++	Care-label v1→v3; RNN → Elk Audio OS VST on RPi4	Strong	Use as your main story
Optimization under budgets	30 ms window, 31.4% CPU on RPi4; Nebula ARM latency benchmarks	Strong (CPU/ARM)	Show your profiling method (section 3)
OCR	Tech-pack digitization; multilingual care-label verification; COVID vitals digit reading	Strong	Tie to Aurora's OCR tools
Detection / recognition	Template + feature matching, fabric defects, garment segmentation	Strong	Classical first, then how you'd add deep learning

JD area	Your best evidence	Strength	How to play it
Registration / geometry	Promptly warp alignment; overlapping-FOV camera array	Partial	Talk homography, calibration, stitching
Code review, tests, docs	Set up CI/CD and code review at MAS; pytest	Strong	One code-review story
SIMD / CUDA / OpenCL	Multi-GPU training only	Gap	Honest line plus a 1-week ramp plan
ViT / VLM	CNNs, Wav2Vec2 benchmarking	Partial	Show you know the deployment issues
French	~A2, in classes	Gap	Never say bilingual

Handling the gaps in the room. Engineers respect "I haven't done X, here's how I'd approach it" more than a stretch. For SIMD and CUDA: "I haven't hand-written kernels. My optimization work has been on CPU and ARM: profiling, memory layout, cutting allocations, keeping a latency budget. I know how I'd map a per-pixel filter onto a GPU, and I can talk it through." Then do it with the answer in section 3. That turns a gap into a demonstration.

Red lines (same as the screen): never cite ">90% F1" (use 0.76, 0.78 or 95%); never mention the Promptly patent; don't claim CUDA/OpenCL kernels, SIMD intrinsics or ViT/VLM projects; don't call your French conversational.

Technical deep dive

The brief C++ test is confirmed. The team's heritage is a C/C++ imaging SDK, so expect modern C++ and memory questions first, then performance, then CV fundamentals. Start with the test drill below; parts A–E are the depth behind it.

The C++ test: drill this first

"Brief" in a 90-minute slot probably means 20–30 minutes. The two likeliest formats:

- **A short quiz.** Matrox's old written test mixed basic C++ and image processing, ran from easy to hard, and candidates were told they weren't expected to finish. Answer the easy items quickly and exactly, and say your reasoning on the hard ones.
- **One small function in a shared editor.** Most likely an image-processing routine (section 4). Clarify, write it simply, test it, then optimize.

Ask Paula this week: is it a quiz or a coding task, which tool or editor, can you compile, and which C++ standard?

Fourteen classic traps. Know each answer and why it happens:

1. Deleting a derived object through a base pointer with no virtual destructor is undefined behaviour; the derived destructor never runs.
2. Object slicing: passing a `Derived` by value as a `Base` copies only the base part, so virtual calls go to the base version.
3. A virtual function called inside a constructor or destructor runs the version of the class currently being built, not the derived override.
4. `uint8_t a = 200, b = 100; auto c = a + b;` makes `c` an `int` equal to 300 (integer promotion). Stored back into a `uint8_t` it wraps to 44. This is why pixel arithmetic accumulates in `int` and clamps.
5. `for (unsigned i = n - 1; i >= 0; --i)` never ends, because an unsigned value is always ⩾ 0. Signed/unsigned comparisons also produce warnings and bugs.
6. `sizeof` of an array parameter gives the pointer size; `sizeof` of a struct includes padding. Order members largest first to shrink it.
7. Returning a reference or pointer to a local variable leaves it dangling.
8. `push_back` can reallocate a `std::vector`, invalidating every iterator, pointer and reference into it. `reserve` up front for hot loops.
9. A class holding a raw owning pointer with the default copy constructor makes two objects free the same memory (double free). Fix it with the rule of 5, or better `unique_ptr`.
10. `std::move` moves nothing; it's a cast to an rvalue. A moved-from object is valid but unspecified.
11. `const int* p` is a pointer to a const int; `int* const p` is a const pointer to an int. Read right to left.
12. A `static` local is initialised once (thread-safe since C++11). A `static` member is shared by all objects. `static` at file scope means internal linkage.
13. Two `shared_ptr`s pointing at each other never free (a cycle). Break the cycle with `weak_ptr`.
14. Overloading vs overriding: overloading is the same name with different parameters, resolved at compile time; overriding replaces a virtual function, resolved at run time. Mark overrides `override`.

Practice: write a 10-line program for each trap you're unsure of and run it in Compiler Explorer (godbolt.org) with `-Wall -fsanitize=address,undefined`. Watching the warning appear sticks better than reading about it.

A. Modern C++ (highest probability)

1. **RAII?** Resources are tied to object lifetime: acquire in the constructor, release in the destructor. That makes code exception-safe and leak-free. Examples: `std::unique_ptr`, `std::lock_guard`, a buffer class that frees on destruction.
2. **unique_ptr vs shared_ptr vs weak_ptr?** `unique_ptr` means single ownership, move-only, zero overhead. `shared_ptr` means reference-counted shared ownership, with an atomic count and a control block, so it's slower. `weak_ptr` observes without owning and breaks reference cycles. Default to `unique_ptr`. Prefer `make_unique` / `make_shared`.
3. **Rule of 0/3/5?** If a class manages a resource, define or delete the destructor, copy constructor, copy assignment, move constructor and move assignment. Better is the rule of zero: hold resources in RAII members and write none of them.
4. **Move semantics?** An rvalue reference (`T&&`) lets you steal a temporary's buffer instead of deep-copying it, which matters for large image buffers. `std::move` is only a cast. Mark move constructors `noexcept` so `std::vector` uses them when it reallocates.
5. **Pointer vs reference?** A reference can't be null or reseated and needs no dereference syntax. Pass large read-only objects as `const T&`, optional ones as a pointer or `std::optional`, and use `std::span` (C++20) for buffers.
6. **Class vs object; struct vs class?** A class is the type and an object is an instance. A struct is public by default, a class is private by default. This came up in a Matrox interview, so answer it in one line.
7. **Virtual functions and cost?** They dispatch through a vtable pointer: an indirect call that is hard to inline and sometimes costs a cache miss. Avoid them in per-pixel inner loops; use templates or CRTP instead. Always give a polymorphic base a virtual destructor.
8. **const correctness, `constexpr`, `inline`?** `const` member functions promise not to change the object. `constexpr` lets the compiler evaluate at compile time. `inline` lets a definition appear in several translation units.
9. **Undefined behaviour you've seen?** Out-of-bounds access, use after free, signed overflow, data races, strict-aliasing violations. Tools: AddressSanitizer, UBSan, ThreadSanitizer, Valgrind.
10. **Templates vs inheritance for a pixel-type-generic filter?** Use templates over the pixel type (uint8, uint16, float) for zero-cost generic code, and type erasure at the API boundary. This is how imaging libraries like the Aurora Imaging Library and OpenCV's `Mat` handle many depths.

11. **Concurrency?** `std::thread`, `std::mutex`, `std::atomic`, `std::condition_variable`. A producer-consumer queue between capture and processing. Avoid false sharing by padding per-thread data to 64 bytes. Use a thread pool, not a thread per frame.
12. **Memory layout: stack vs heap; AoS vs SoA?** The stack is fast and limited; the heap is flexible but allocation is slow. In hot loops, preallocate and reuse buffers. Structure-of-arrays vectorises better than array-of-structures.

Debugging (reported at Matrox): if asked how you'd find a crash that happens only in release builds, list uninitialised variables, UB the optimiser exploits, and races. Reproduce with `-O2 -g`, run the sanitizers, then bisect.

B. Performance and optimization (the core of this job)

"How do you optimize an algorithm for a target platform?" Answer as a method:

1. Set the budget: latency per frame, throughput, power, memory, accuracy floor.
2. Measure first: a profiler (perf, VTune, Nsight, gprof) on representative data; find the hotspot.
3. Algorithm before micro-optimization: lower complexity, separable filters, integral images, image pyramids (coarse-to-fine), early rejection, a smaller ROI.
4. Memory: contiguous access, row-major loops, tiling for cache, no per-frame allocations, aligned buffers.
5. Parallelism: multi-core (OpenMP, a thread pool, tile per thread), SIMD (SSE/AVX on x86, NEON on ARM), GPU (CUDA/OpenCL) when the arithmetic per byte justifies the transfer.
6. Precision: fixed-point or int8 where accuracy allows; quantised neural networks.
7. Validate: bit-exact or tolerance tests against the reference implementation, plus a benchmark kept in CI so regressions show up.

Your evidence: on the RPi4 you held a 30 ms window at 31.4% CPU, against 1038 ms for DTW. Nebula has per-feature ARM latency benchmarks.

SIMD in one paragraph (know this cold): one instruction runs on 4–32 values: SSE is 128-bit, AVX2 256-bit, AVX-512 512-bit; NEON on ARM is 128-bit. With uint8 pixels, one AVX2 register holds 32. The compiler auto-vectorises simple loops at `-O3` if there's no aliasing (`__restrict`), the data is contiguous and the loop has no branches. Intrinsics such as `_mm256_add_epi8` give you control. Saturating arithmetic (`_mm256_adds_epu8`) avoids overflow for pixels. Handle the leftover tail elements with a scalar loop.

GPU / CUDA in one paragraph: threads are grouped into blocks and warps of 32. Map one thread to one pixel. Coalesce global memory reads (neighbouring threads read neighbouring addresses), and use shared memory for the halo of a convolution tile. Avoid divergent branches within a warp. Host-to-device transfer over PCIe often dominates, so

batch work, use pinned memory and streams to overlap copy with compute, and keep data on the GPU across pipeline stages. A GPU wins when the work is large and parallel; small images on a camera may run faster on the CPU with NEON.

Likely question: "Make a 5×5 Gaussian blur fast." It's separable: two 1D passes, 10 multiplies per pixel instead of 25. Use fixed-point integer weights. Vectorise the horizontal pass, process the vertical pass row by row so it stays cache-friendly, split rows across threads, and handle borders with replicate or reflect. For large sigma, use a box-filter approximation (3 passes) or a sliding sum so cost doesn't depend on kernel size.

Likely question: "Speed up template matching." Normalised cross-correlation (NCC) computed with an FFT for large templates. Integral images give the window sums and sums of squares. Search a coarse-to-fine pyramid. Restrict the search to a region of interest. Reject early on a low-resolution score. Parallelise across candidate positions.

C. Classical computer vision (Aurora Imaging Library territory)

- **Filtering:** convolution vs correlation; Gaussian, median (good for salt-and-pepper noise), bilateral (edge-preserving); Sobel and Scharr gradients.
- **Edges and features:** Canny steps (smooth → gradient → non-max suppression → hysteresis); Harris corners; SIFT, ORB and AKAZE (ORB is binary, fast and uses Hamming matching); RANSAC for robust fitting.
- **Segmentation and blobs:** Otsu and adaptive thresholding; morphology (erode, dilate, open removes specks, close fills holes); connected components and blob analysis (area, centroid, moments); watershed.
- **Pattern matching:** NCC (invariant to linear brightness changes); geometric or edge-based model finding, which is robust to occlusion and rotation and is what Matrox's model finder does; Hough transform for lines and circles.
- **Measurement and metrology:** sub-pixel edge location (fit a parabola to the gradient peak), caliper tools, pixel-to-mm calibration.
- **OCR:** classical pipeline is binarise → deskew → segment characters → classify (template or CNN) → verify against a font or syntax rule. Modern approach: a CRNN with CTC, or a transformer reader. Industrial difficulties: dot-matrix print, low contrast, curved surfaces. Use your care-label and vitals stories here.
- **Colour:** HSV or Lab for colour segmentation; white balance; why RGB distances mislead.

D. Geometry, calibration, 3D (Photoneo and "multiple view geometry")

- **Pinhole model:** x = K [R | t] X. K holds focal lengths, principal point and skew. Lens distortion is radial (k1, k2, k3) plus tangential (p1, p2).

- **Calibration:** Zhang's method with a checkerboard in several poses. Estimate homographies, solve for the intrinsics, then refine everything with Levenberg–Marquardt to minimise reprojection error (aim for under 0.5 px).
- **Homography:** a 3×3 matrix with 8 degrees of freedom mapping one plane to another, needing 4+ point pairs. Solve with DLT plus RANSAC. Uses: perspective correction, stitching. Promptly's alignment is your bridge here.
- **Epipolar geometry:** the fundamental matrix F (uncalibrated, 7 DOF, 8-point algorithm) and the essential matrix $E = K^T F K$ (calibrated, gives R and t up to scale). Rectify stereo pairs, then disparity gives depth $Z = f \cdot B/d$.
- **Structured light (Photoneo's domain):** project known patterns, decode the correspondences, triangulate. Output is a point cloud.
- **Point clouds:** ICP registration, normals, plane fitting with RANSAC, voxel downsampling, nearest neighbours with a k-d tree.
- **SLAM / SfM in one line:** match features across views → estimate poses → triangulate → bundle adjustment. Say honestly that you know the pipeline but haven't built a SLAM system.

E. Deep learning on devices

- **Deployment path:** PyTorch → ONNX → TensorRT (NVIDIA) or OpenVINO (Intel) → an on-camera accelerator SDK. Aurora's deep-learning tools run on NVIDIA and Intel GPUs.
- **Optimization:** int8 post-training quantisation with calibration data, or quantisation-aware training; pruning; distillation; operator fusion; batch size 1 for latency.
- **Detection basics:** IoU, NMS, anchor vs anchor-free (YOLO family), mAP; segmentation with U-Net.
- **Anomaly detection (Aurora has this):** train only on good parts, because defects are rare. Approaches: an autoencoder's reconstruction error, or feature-embedding methods such as PatchCore and PaDiM (distance from a memory bank of normal patches). Threshold on a held-out set. Your fabric defect work fits here.
- **ViT vs CNN:** a ViT splits the image into patches and runs self-attention, giving a global receptive field. It needs more data or pre-training and attention costs $O(n^2)$ in the number of tokens, so it's heavier on edge devices. CNNs have locality bias and are cheaper.
- **VLMs:** an image encoder plus a language model, useful for zero-shot inspection prompts or labelling data. They are too heavy for on-camera inference today, so the realistic use is offline labelling or a server-side assistant.

- **Classical vs deep learning?** Classical when the conditions are controlled, data is scarce, results must be explainable, or the cycle time is tight. Deep learning when appearance varies a lot. In industry you often combine them: classical for locating and aligning, deep learning for classifying.

Live coding

If the brief C++ test is a coding task rather than a quiz, it will most likely be a small image-processing function in C++ on a shared editor or a whiteboard, not LeetCode-hard puzzles. They're judging clean modern C++, border handling, integer overflow, complexity, and whether you test your own code.

How to run any coding question (5 steps, out loud):

1. Clarify: pixel type (uint8? float?), layout (row-major, stride or pitch?), channels, border policy, in-place or out-of-place, image size.
2. Write the signature first, with `const` inputs and a stride parameter.
3. Write the simple correct version, then state its complexity in time and memory.
4. Walk through a 3×3 example by hand, including edge cases: an empty image, a 1-pixel-wide image, all-255 values.
5. Offer optimizations without being asked: separability, integral image, SIMD, threads.

Practice set, most likely first. Do each in under 20 minutes, by hand, before the interview.

#	Problem	Key insight	Trap
1	3×3 convolution / box blur on uint8	Accumulate in int32, clamp to 0–255	Overflow; borders; stride ≠ width
2	Box filter of any size in O(1) per pixel	Integral image (summed-area table)	Off-by-one at the image edge; int64 for large images
3	Connected components labelling	BFS/DFS or two-pass with union-find	Recursion depth on big blobs: use an explicit stack
4	Histogram + Otsu threshold	Maximise between-class variance in one pass over 256 bins	Division by zero when one class is empty
5	Bilinear resize / image rotation	Inverse mapping from destination to source	Pixel-centre convention (+0.5)

#	Problem	Key insight	Trap
6	Erode / dilate with a 3×3 element	Min/max filter; separable for rectangles	Border values (pad with 255 for erode)
7	NCC template matching	Subtract means, divide by the standard deviations	Zero-variance windows
8	Rotate an N×N image 90° in place	Transpose, then reverse each row	Doing it in place
9	Thread-safe bounded queue (capture → process)	mutex + 2 condition variables	Spurious wakeups: wait with a predicate
10	Implement a simple `unique_ptr` or image buffer class	Rule of 5, move-only	Self-move; `noexcept`

Worked example (#2). Learn it; it shows off several skills at once:

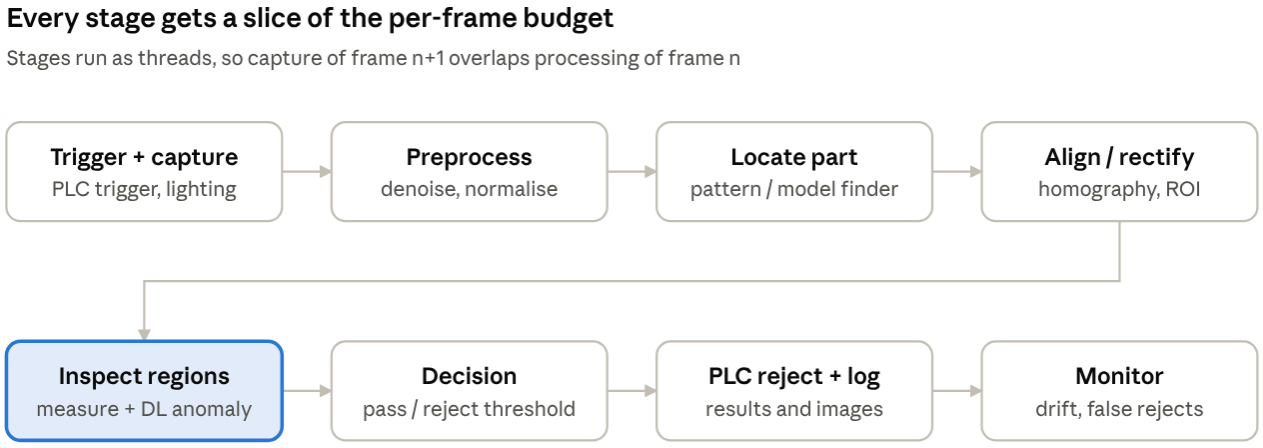
```
// Box mean filter, radius r, O(1) per pixel via integral image.
void boxMean(const uint8_t* src, uint8_t* dst, int w, int h, int stride, int
r) {
    std::vector<uint32_t> I((w + 1) * (h + 1), 0);   // padded: row/col 0 are
zeros
    for (int y = 0; y < h; ++y) {
        uint32_t rowSum = 0;
        for (int x = 0; x < w; ++x) {
            rowSum += src[y * stride + x];
            I[(y + 1) * (w + 1) + (x + 1)] = I[y * (w + 1) + (x + 1)] +
rowSum;
        }
    }
    for (int y = 0; y < h; ++y) {
        int y0 = std::max(0, y - r), y1 = std::min(h - 1, y + r);
        for (int x = 0; x < w; ++x) {
            int x0 = std::max(0, x - r), x1 = std::min(w - 1, x + r);
            uint32_t s = I[(y1 + 1) * (w + 1) + (x1 + 1)] - I[y0 * (w + 1) +
(x1 + 1)]
                       - I[(y1 + 1) * (w + 1) + x0] + I[y0 * (w + 1) + x0];
            int n = (y1 - y0 + 1) * (x1 - x0 + 1);
            dst[y * stride + x] = static_cast<uint8_t>((s + n / 2) / n);  //
rounded
        }
    }
}
```

Points to say aloud: uint32 is safe up to 16.8M pixels of 255 (larger images need uint64). Borders shrink the window, so it stays unbiased. The cost doesn't depend on r. Next steps would be removing the integer division by precomputing reciprocals for interior pixels, splitting rows across threads, or a sliding-sum version that needs no extra full-size buffer.

Python fallback. If they let you prototype in Python first, do it in NumPy (vectorised, no loops), then say how you'd port it. That mirrors the job description's "prototype → product".

Design scenario

No design round is listed, but Jean-Sebastien or Sylvain may still ask an open question such as "design an inspection system for X on our smart camera" or "here's a prototype algorithm; how do you get it into the product?" Both reward the same structure: budget first, pipeline second, validation third.



inspection pipeline · 8 stages from trigger to monitoring

Draw this on the whiteboard as you answer. The highlighted stage is where classical measurement and a deep-learning anomaly model meet, so profile it first.

Framework (say the headers aloud so the panel can follow):

1. **Requirements:** parts per minute, so the cycle time per frame; defect size compared with the field of view, which sets resolution; lighting; false-reject versus false-accept cost; the target hardware (x86 PC, NVIDIA GPU, ARM smart camera with an accelerator).
2. **Optics and capture first:** lighting solves half the problem (backlight for silhouettes, dome light for shiny parts, low-angle light for surface defects). Trigger from an encoder or PLC. Fix the exposure.

3. **Pipeline:** acquisition → preprocessing → locate the part (pattern or model finder) → align or rectify → inspect each region (classical measurement plus a deep-learning classifier or anomaly detector) → decision → I/O to the PLC reject, with results logged.
4. **Budget allocation:** for example, at 10 parts per second you have 100 ms per part, so give each stage a millisecond allowance and profile against it. Run the stages as a pipeline on separate threads so capture of frame n+1 overlaps processing of frame n.
5. **Productization:** a clean API, configuration separate from code, deterministic results, versioned models, tests that are bit-exact or within tolerance against the Python reference, a performance benchmark in CI, and an architecture doc.
6. **Validation:** a golden dataset with labelled defects; report false-reject and false-accept rates at the chosen threshold; drift monitoring in production (lighting or supplier changes).

Map it to your care-label system. It's the same design, already shipped: v1 flatbed scanner (prototype), v2 industrial camera (real capture), v3 conveyor with PLC reject (production I/O), fully local processing for compliance. Walk through it in this structure and the panel will see you have done this job before.

"Port this Python prototype to C++ for the camera." Freeze the reference outputs first (golden files). Port module by module behind the same interface. Unit-test each against the reference. Profile on the target, not your laptop. Remove allocations and copies in the hot path. Choose the precision (float → fixed-point or int8). Only then reach for SIMD or the accelerator. Document what you changed and why.

Project deep-dive and behavioural

Pick **one** project to go deep on: the care-label QC system. It covers the whole job: prototype to product, classical CV, OCR-style verification, hardware I/O, and a measured outcome. Keep Promptly and the RPi4 work as second and third stories.

Deep-dive script: care-label QC (5 minutes, then let them dig)

- **Problem:** labels in several languages had to be checked against each order's spec by hand, 15–20 minutes per label, with errors reaching customers such as major brands.
- **v1 (prototype):** flatbed scanner, OpenCV template and feature matching against the approved artwork, with an explainable overlay showing where a label differed.
- **v2:** an industrial camera replaced the scanner, which brought new problems: lighting, perspective and focus. Add how you handled alignment (registration before comparison).
- **v3 (product):** a conveyor with a PLC-driven reject and all processing kept on site for compliance.

- **Result:** 1–2 minutes per batch instead of 15–20 minutes per label; up to +300% throughput.
- **What you'd do now:** a learned anomaly detector for print defects alongside the template match, plus an OCR verification step. Both are Aurora capabilities, so name them.

Prepare to be challenged on: false-reject rate and how you tuned the thresholds; what broke in v2; how you tested it; latency per label; what was in C++ and what was in Python.

Behavioural stories in STAR form. One story can answer several questions.

Likely question	Story	Key beats
Prototype to production	Care-label v1→v3	Three iterations, PLC integration, measured outcome
Hardest technical problem / ambiguity	Promptly alignment	Camera-captured garment geometry → per-side warp so prints align at the seams; 4–6 min per garment; built the RIP integration yourself; now a product in 4 countries. Don't mention the patent
Optimizing under a hard budget	RPi4 pattern detection	30 ms window; 14 ms at 31.4% CPU against 1038 ms for DTW; F1 0.76 against 0.65
Improving team practice / code review	Introducing CI/CD and code review at MAS	Why, resistance, how you made it easy, what it caught
Cross-functional work	Aeoon printer integration	Reverse-engineered PLC trigger points and RIP hooks; 3 closed-loop subsystems in C++ with mechanical and print teams
Real-time constraint with physical consequences	Loom-side fabric defects	Microscopic camera array with overlapping FOV across 60-inch rolls; 5–6 s alert budget; pilot cut operators 50%

Likely question	Story	Key beats
Fast delivery under pressure	COVID remote vitals	Webcams retrofitted to SpO_2 and BP monitors, image processing → dashboard in hospital wards; GMOA recognition
Large collaboration	MUSMET (EU, 12 partners)	Synchronising BCI, XR and audio data streams with external partners
Disagreement with a colleague	Prepare one from MAS or the postdoc	Disagreed on data, ran a test, accepted the result
A failure	Prepare one honest story	What broke, what you changed in your process

Questions about your profile, with short answers:

- *Why go back from audio to vision?* "Same problem shape: real-time detection on constrained hardware. Spectrograms are images, and I used SSIM from vision in published work. Vision is where my industrial career started."
- *You have a PhD; will you be bored with productization?* "Productization is the part I like most. Everything I published ran on hardware."
- *How do you use AI coding tools?* (The job description mentions GenAI tools.) "Daily, for boilerplate, test scaffolding and exploring APIs. I review every line, and I never let them write performance-critical code unprofiled."

Questions to ask, salary and logistics

Ask one or two questions of each person, aimed at what they own. Learning about the role is half the purpose of this session, so good questions count as much as your answers.

For Jean-Sebastien (supervisor, CV software development)

1. What does the path from a research prototype to an Aurora Imaging Library release look like today: who writes the prototype, who ports it, and how is it validated?
2. Which targets give you the most trouble right now: x86, NVIDIA, ARM smart cameras, or the on-camera accelerators?
3. How do you test performance: is there a benchmark suite in CI, and how are regressions caught?
4. What C++ standard and toolchain is the codebase on, and how much legacy C remains?

5. How are the deep-learning tools (anomaly detection, OCR) built and deployed: own runtime, ONNX, TensorRT, OpenVINO?
6. How does the 3D work from Photoneo connect with the 2D library team?

For Sylvain (senior manager, CV & AI)

1. What would a great first six months look like in this role?
2. How has the team changed over the past year, and how big is it now? (A polite route to the 2026 restructuring.)
3. How are priorities set between new algorithms and productizing existing ones?
4. Is there flexibility on level given my experience? Ask this only if the interview is going well and the manager hasn't raised it first.

For Paula (process)

1. How many rounds follow this one, and what are they?
2. What's the timeline to a decision?

Salary. If it comes up again, repeat the same anchor: about $130K base plus the 10% bonus, flexible on the overall package including relocation. Don't drop the number in a second round. If they mention level, say: "I'm happy to be assessed for whichever level fits the scope."

Signals to listen for: team headcount trending down; vague answers about the roadmap; "we're mostly maintaining"; all decisions made outside Montreal. None of these is a deal-breaker, but they change what you ask for at the offer stage.

Close (30 seconds): "Thanks. This sounds like the work I want to be doing: turning vision algorithms into fast, tested product code on real cameras. Is there anything about my background you'd like me to clarify?" Thank all three by name.

Prep plan and day-of checklist

You have seven days. The C++ test gets the most practice time because it's the only part with right and wrong answers.

Wed 21 Oct (today)

- [ ] Reply to Paula to confirm, and ask about the C++ test format: quiz or coding task, which tool, can you compile, which C++ standard
- [ ] Look up Jean-Sebastien and Sylvain on LinkedIn; note their tenure and background
- [ ] Read the C++ test drill (section 3)

Thu 22 Oct

- [ ] Revise section 3A (modern C++) and run each of the 14 traps you're unsure of in Compiler Explorer
- [ ] Revise smart pointers, rule of 5 and move semantics until you can explain each in 30 seconds
- [ ] Rehearse the four 60-second answers in the panel background section (laser triangulation, photometric stereo, OCR for a new font, few labels)

Fri 23 Oct

- [ ] Write practice problems 1–5 (section 4), 20 minutes each, in a plain editor without autocomplete
- [ ] Rebuild the integral-image box filter from memory and test it on a 5×5 example

Sat 24 – Sun 25 Oct

- [ ] Write practice problems 6–10 (section 4)
- [ ] Read the Aurora Imaging Library product pages: model finder, OCR, deep-learning anomaly detection, 3D
- [ ] Revise SIMD, the GPU basics and the geometry notes (sections 3B and 3D)

Mon 26 Oct

- [ ] Rehearse the 2-minute pitch and the 5-minute care-label deep-dive aloud, recorded
- [ ] Write out your STAR stories, including a failure and a disagreement (section 6)
- [ ] Pick one question for each person (section 7)

Tue 27 Oct

- [ ] Do a full 90-minute mock: pitch, deep-dive, a 25-minute timed C++ quiz drawn from the traps and problems, then your questions
- [ ] Test the video link, camera, microphone and screen sharing; keep Compiler Explorer and a plain editor ready
- [ ] Check whether Zebra's Q3 results are out, for one line of small talk
- [ ] Settle your salary line (section 7) in case Paula raises it

Wed 28 Oct, 1:15–2:45 pm ET

- [ ] Be online by 1:00 pm with the job description, this guide, the CV you submitted, paper and a pen
- [ ] Within 24 hours, send a thank-you email to PE5556@zebra.com, Jean-Sebastien.Lemieux@zebra.com and Sylvain.Chapleau@zebra.com, each mentioning one specific point from your conversation

Sources

The interview-format notes come from search-result summaries of anonymous candidate reports; the pages themselves could not be opened from this session. Treat them as signals, not facts.

- Zebra computer vision engineer interview reports — Glassdoor
- Zebra software developer interview reports — Glassdoor
- Matrox Montreal interview, Nov 2021 — Glassdoor
- Matrox written C++ and image-processing test — Glassdoor
- Zebra software engineer interview guide — Interview Query
- Zebra interview guide — Dataford
- Job description JR103160 and company facts: your Recruiter Screen Guide (7 Oct 2026)
- Your project facts: ai-job-search knowledge graph. Date, panel and format: Zebra's Round 1 interview confirmation email

Panel background:

- Sylvain Chapleau — LinkedIn and ZoomInfo
- US 8,787,660 — Automatic font definition and US 8,620,078 — Determining a class associated with an image
- Sylvain Chapleau's patents — Justia
- US 11,120,294 — 3D profiling of an object and US 12,322,128 — Photometric stereo reconstruction
- Zebra Research Chair in Computer Vision — ÉTS Montréal

Found through web search; the pages themselves couldn't be opened from this session.